ADR 集合

ADR-001：采用 MVC + 分层架构作为系统整体架构风格

状态

已接受

背景

团队需要在 10 周内完成一个包含用户管理、需求发布、订单流程、评价信用等完整业务闭环的约拍服务平台。团队成员为本科生，具备基本的 Web 开发能力，但缺乏分布式系统和生产环境运维经验。需要在"代码结构清晰"和"开发效率"之间找到平衡点。

决策

采用 MVC + 分层架构（Controller → Service → Domain → Repository）作为整体架构风格，以模块化单体方式组织代码，在订单完成、通知推送等关键节点引入 Spring Event 实现轻量事件驱动。

理由

1. MVC 框架（Spring Boot）提供成熟的 Web 基础设施，免去路由、参数绑定、安全等重复建设。
2. 严格四层分层让复杂的订单状态机和档期校验逻辑有明确的"归属地"，不会散落在 Controller 中。
3. 单体部署消除了分布式系统带来的网络通信、分布式事务和联调成本。
4. 模块化划分保留了未来向微服务演进的能力。

后果

- 正面影响：开发效率高，调试方便（一个进程内，报错即可定位），4 人团队可按模块并行开发。
- 负面影响：单体应用在极端高并发场景下无法针对单个模块独立扩容。
- 需要关注的风险：团队需自律遵守分层规范，避免在 Controller 中直接写业务逻辑导致分层退化。建议通过代码审查（Code Review）机制保障。

AI 辅助记录

- AI 初稿内容摘要：AI 建议采用分层架构 + MVC，还建议引入事件驱动解耦订单完成后的通知和评价逻辑。
- 人工修订内容：删掉了 AI 建议的"引入 Spring Cloud Gateway 做 API 网关"和"未来可拆分为微服务独立部署"等内容。AI 在回答中混入了微服务的技术栈（服务注册、网关、分布式事务），但我们连 Docker 都不太熟，根本用不上这些。
- 修订理由：AI 虽然嘴上说"不建议微服务"，但给出的代码示例和架构图里仍然出现了网关、服务注册等微服务组件，前后矛盾。我们判断这是 AI 的惯性输出——它训练数据里大量的架构文章都在讲微服务，所以即使限定了"单体"也会往微服务靠。团队讨论后决定把所有微服务相关内容全部删除，只保留纯单体分层方案。

OBJ

ADR-002：图片文件采用对象存储，数据库仅存 URL

状态

已接受

背景

约拍平台涉及两类图片存储需求：一是服务方公开的作品集（样片/客片），二是订单交付的私有作品（原片/精修片）。平台需要支持图片的上传、在线预览和下载。如果将图片以 BLOB 形式存储在 MySQL 中，将导致数据库体积急剧膨胀，严重影响查询性能。

决策

图片文件统一存储在文件系统或对象存储中，MySQL 数据库仅保存图片的 URL 地址和元数据（文件名、上传时间、文件大小、分类标识）。MVP 阶段采用本地文件存储或 MinIO，正式环境可平滑切换至云对象存储（如阿里云 OSS）。

理由

1. 数据库只存 URL 元数据，保持轻量，不影响订单、用户等核心业务表的查询性能。
2. 存储层与业务层解耦——通过统一的文件存储接口抽象，后续更换存储方案（本地→MinIO→云 OSS）无需修改业务代码。
3. 课程项目阶段使用本地存储或 MinIO 可免去云服务账号和费用，降低环境搭建门槛。
4. 云对象存储天然支持 CDN 加速和图片处理（裁剪、压缩、缩略图），后续切换即可获得这些能力。

后果

- 正面影响：系统性能不受图片数量增长影响；前端可直接通过 URL 访问图片，减少后端带宽压力。
- 负面影响：本地存储在多机部署时需要共享文件目录；MinIO 需要额外的 Docker 容器。
- 需要关注的风险：需设计统一的文件存储接口（如 FileStorageService），屏蔽底层存储实现差异，确保后续切换云存储时业务代码零修改。

AI 辅助记录

- AI 初稿内容摘要：AI 直接推荐使用阿里云 OSS 存储图片，并给出了一段使用 OSS SDK 上传文件的 Java 代码，还提到"配置 STS 临时凭证实现前端直传"。
- 人工修订内容：把"阿里云 OSS"改成了"先用本地文件存储，后续可以换"。因为我们 4 个人都没有阿里云账号，也没有钱买云服务。AI 给的那段 STS 临时凭证代码我们完全看不懂，问 AI 解释了一遍还是不明白。另外 AI 把作品集图片和订单交付图片当成同一种东西处理，但实际上作品集是公开的、交付作品是只有买家能看的，权限完全不一样。
- 修订理由：AI 默认我们是有云服务预算的正式团队，给出的方案超出了课程项目的实际条件。先用最简单的本地文件存储把功能跑通，等有需要了再换，这样更现实。

OBJ

ADR-003：订单状态使用枚举统一管理，禁止散写 if-else

状态

已接受

背景

约拍平台的订单有很多状态：待支付、已支付、拍摄中、待交付、已交付、已完成、已取消、仲裁中。每个状态只能转到特定的下一个状态（比如"已取消"的订单不能再变成"已支付"）。如果在代码里到处用 if-else 判断当前状态，很容易漏掉某个分支，导致出现不该出现的状态转换 Bug。

决策

把所有订单状态写成一个枚举类（OrderStatus），在 Order 实体里写好每个状态能转到哪些状态。改状态的时候统一调用 Order 的方法（比如 order.pay()、order.cancel()），方法内部自动检查当前状态是否允许这个操作，不允许就直接报错。

理由

1. 所有状态转换规则集中在一个地方，不会分散在各个 Controller 和 Service 里，改起来方便。
2. 如果有人写了一段代码让"已取消"的订单变成"已支付"，运行时会直接报错，不会悄悄执行。
3. 后面要加新状态（比如"待返修"），只需要改枚举和 Order 类，不用满项目找 if-else。

后果

- 正面影响：订单逻辑集中、清晰，不容易出现"幽灵状态"的 Bug。
- 负面影响：需要团队统一认识，改订单状态必须通过 Order 的方法，不能直接 setStatus()。
- 需要关注的风险：要在项目开始前约定好规范，否则有人图省事直接改数据库字段就绕过了检查。

AI 辅助记录

- AI 初稿内容摘要：AI 给了一个订单状态枚举和状态机的代码示例，看起来写得挺好。但仔细看发现它只写了 6 个状态：待支付→已支付→拍摄中→交付中→已完成→已评价，是一条直线走到底的"顺利流程"。
- 人工修订内容：我们对照 P1 阶段写的用例（UC-05 下单支付、UC-08 成片质量申诉），发现 AI 漏掉了"已取消""仲裁中""待返修"这三个状态。这三个状态对应的是用户取消订单、成片不满意发起投诉、管理员

判定需要返修这些情况。如果不加这些状态，上线后用户取消订单都没法处理。另外 AI 也没考虑"7 天未确认自动完成"这个超时逻辑。

• 修订理由：AI 只考虑了一切顺利的情况，完全忽略了出问题怎么办。但实际上用户调研里反映最多的痛点就是"跑单""成片不满意没法退"，这些异常流程反而是最重要的。这说明 AI 在生成状态设计时，不会主动去看我们之前写的需求文档和用例描述，需要人工把这些上下文喂给它或者自己补上。

ADR-004：前后端通信采用 RESTful API，即时消息 MVP 阶段先用轮询

状态

已接受

背景

约拍平台大部分功能（浏览作品集、筛选服务方、提交订单等）用普通的 HTTP 请求就够了。但平台还有一对一聊天和订单状态通知的需求，理论上需要服务器主动给客户端推消息。

决策

常规业务接口采用 RESTful API。即时消息和通知在 MVP 阶段先用 HTTP 轮询（前端每隔几秒请求一次新消息），架构上预留 WebSocket 升级空间，后续有精力再替换。

理由

1. RESTful API 是团队最熟悉的通信方式，能覆盖 90% 以上的业务场景。
2. WebSocket 虽然体验更好，但团队没有人实际用过，涉及连接管理、断线重连、心跳检测等问题，学习和调试成本较高。
3. 轮询方案虽然不够实时（有几秒延迟），但对课程项目来说完全够用，不影响核心业务演示。
4. 消息接口设计上保持和 WebSocket 兼容（返回 JSON 格式的消息列表），后续切换只改通信层，不改业务逻辑。

后果

• 正面影响：技术风险低，团队可以把精力集中在核心业务（订单流程）上，而不是花时间调 WebSocket 连接问题。

• 负面影响：消息有几秒的延迟，不如真正的即时通讯流畅。

• 需要关注的风险：轮询频率不能太高（建议 3-5 秒），否则对服务器有压力；也不能太低，否则用户感觉消息"不来"。

AI 辅助记录

• AI 初稿内容摘要：AI 建议直接使用 WebSocket 实现即时通讯，给了一段 Spring WebSocket 配置代码和前端连接代码，并说"Spring Boot 原生支持 WebSocket，集成很简单"。

• 人工修订内容：我们试着按 AI 给的代码跑了一下，发现配置比 AI 说的复杂得多——CORS 跨域要额外配置、Flutter 端的 WebSocket 库和 Spring 的消息格式对不上、断线之后不会自动重连。光是让 WebSocket 跑通就花了一个下午还没成功。团队讨论后决定 MVP 先用轮询，把 WebSocket 放到后面有时间再做。

• 修订理由：AI 说"集成很简单"是站在它的角度——它见过很多 WebSocket 的代码。但对我们来说，光是理解 STOMP 协议、消息代理这些概念就要花不少时间。课程项目的核心是跑通订单闭环，不是做一个完美的聊天系统。先用能跑通的方案，后面再优化。